

Parentheses Checker Using Stack in C

C Program:

```
#include <stdio.h>
#include <string.h>

char stack[100];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }

int isMatching(char a, char b) {
    return (a == '(' && b == ')') ||
           (a == '{' && b == '}') ||
           (a == '[' && b == ']');
}

int isBalanced(char exp[]) {
    for (int i = 0; i < strlen(exp); i++) {
        if (exp[i] == '(' || exp[i] == '{' || exp[i] == '[')
            push(exp[i]);
        else if (exp[i] == ')' || exp[i] == '}' || exp[i] == ']') {
            if (top == -1)
                return 0;
            if (!isMatching(pop(), exp[i]))
                return 0;
        }
    }
    return (top == -1);
}

int main() {
    char exp[100];
    printf("Enter expression: ");
    fgets(exp, sizeof(exp), stdin);
    exp[strcspn(exp, "\n")] = 0;

    if (isBalanced(exp))
        printf("Expression is Balanced\n");
    else
        printf("Expression is Not Balanced\n");
    return 0;
}
```

Explanation:

1. Stack Implementation: 'stack' array stores opening brackets and 'top' points to the top element. 2. push() and pop() functions are used to insert and remove elements from the stack. 3. isMatching() checks whether an opening bracket matches a closing bracket. 4. isBalanced() scans the expression: - Pushes opening brackets onto the stack. - On encountering closing brackets, pops from stack and checks match. - If mismatch occurs or stack is not empty at end, expression is not

balanced. 5. Safe input: `fgets()` is used instead of deprecated `gets()`. The newline character added by `fgets()` is removed. 6. Sample Runs: - Input: $\{(A+B)*(C+D)\}$ -> Output: Expression is Balanced - Input: $(A+B]*(C+D)$ -> Output: Expression is Not Balanced